

DAY 6 | PostgreSQL 18 Performance Tuning

Step-by-step instructor lab | Patroni three-node GCP cluster | PgBouncer on pg-node-2

Duration: 6-8 hours. Run shell blocks on the named VM; run SQL blocks inside psql. Each line is real PDF text for selection and copying.

Environment and safety

Nodes: pg-node-1 10.10.0.11, pg-node-2 10.10.0.12, pg-node-3 10.10.0.13. Patroni scope: postgres-ha. Your leader may change: discover it at the start of each lab. Use the disposable database perf_lab; never initialize pgbench in banking. All benchmarks generate real writes, WAL and replication traffic. Complete backups first and schedule workload tests when the cluster is otherwise idle.

Prompts: Shell examples beginning with sudo run in an Ubuntu shell; SQL examples beginning SELECT/SHOW/EXPLAIN run in psql. The following commands use local peer access via sudo -u postgres on the current primary, avoiding password literals. Do not run writes on a replica.

1. Locate the primary and capture a baseline

1.1 Run on any node (pg-node-1 is convenient)

```
sudo -u postgres /opt/patroni/bin/patronictl -c /etc/patroni/patroni.yml list
sudo -u postgres /opt/patroni/bin/patronictl -c /etc/patroni/patroni.yml show-config postgres-ha
```

Read the Leader row. Open SSH to that node for steps 1-9. Verify locally:

```
sudo -u postgres psql -d postgres -X -c "SELECT inet_server_addr(), pg_is_in_recovery();"
sudo -u postgres psql -d postgres -X -c "SELECT version();"
sudo -u postgres psql -d postgres -X -c "SHOW data_directory;"
```

Expected: pg_is_in_recovery = f. If t, move to the current leader. Record the starting settings and metrics; snapshots are cumulative, so compare differences across runs.

```
sudo -u postgres psql -d postgres -X <<'SQL'
SELECT name,setting,unit,pending_restart FROM pg_settings
WHERE name IN (
'shared_buffers','work_mem','maintenance_work_mem',
'effective_cache_size','max_wal_size','checkpoint_timeout',
'checkpoint_completion_target','wal_compression','autovacuum',
'track_io_timing') ORDER BY name;
SQL

sudo -u postgres psql -d postgres -X <<'SQL'
SELECT num_timed,num_requested,num_done,write_time,sync_time,buffers_written FROM pg_stat_checkpoint;
SQL

sudo -u postgres psql -d postgres -X <<'SQL'
SELECT wal_records,wal_fpi,pg_size_pretty(wal_bytes) AS wal_bytes,wal_buffers_full FROM pg_stat_wal;
SQL
```

PostgreSQL 18 calls the checkpoint counter num_timed and num_requested. Inspect the view with \d pg_stat_checkpoint if working against a different major version.

2. Build a reproducible query-planning exercise

2.1 Create an isolated database and sample data on the primary

```
sudo -u postgres createdb perf_lab
sudo -u postgres psql -d perf_lab -X <<'SQL'
CREATE TABLE lab_orders (
    order_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id integer NOT NULL,
    order_date date NOT NULL,
    status text NOT NULL,
    amount numeric(12,2) NOT NULL
);
INSERT INTO lab_orders (customer_id,order_date,status,amount)
SELECT (g % 10000) + 1, DATE '2025-01-01' + (g % 365),
       CASE WHEN g % 10 = 0 THEN 'PENDING' ELSE 'PAID' END,
       (g % 5000 + 100)::numeric / 10
FROM generate_series(1,300000) AS g;
ANALYZE lab_orders;
SELECT count(*) FROM lab_orders;
SQL
```

Expected count: 300000. Use smaller generate_series(1,50000) if disk is constrained. Plans and timings vary by VM, cache state, and settings.

2.2 Observe estimates, actual rows, buffers and timing

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS)
SELECT * FROM lab_orders
WHERE customer_id = 427 AND status = 'PAID'
ORDER BY order_date DESC LIMIT 20;
SQL
```

Read plan nodes bottom-up. Compare estimated rows with actual rows; inspect Seq Scan, Sort, Execution Time, shared hit/read and temp read/written. EXPLAIN ANALYZE executes the statement: never apply it to destructive SQL on real data without a rollback plan.

3. Index and query optimization

3.1 Add an index aligned with filter and sort

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
CREATE INDEX lab_orders_customer_date_idx
ON lab_orders (customer_id, order_date DESC);
ANALYZE lab_orders;
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM lab_orders
WHERE customer_id = 427 AND status = 'PAID'
ORDER BY order_date DESC LIMIT 20;
SQL
```

Compare scan, sort, buffers and time with step 2. For a real busy table, plan CREATE INDEX CONCURRENTLY outside a transaction and budget extra I/O. An index adds write cost and disk use; a sequential scan can still be optimal for large result sets.

3.2 Compare a non-sargable predicate and an indexable range

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
CREATE INDEX lab_orders_date_idx ON lab_orders(order_date);
ANALYZE lab_orders;
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM lab_orders
WHERE EXTRACT(YEAR FROM order_date) = 2025;
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM lab_orders
WHERE order_date >= DATE '2025-01-01'
      AND order_date < DATE '2026-01-01';
SQL
```

Both match nearly all rows in this dataset, so either may scan sequentially. To see selective behavior, change the range to one day and compare; avoid claiming that every index use is faster. Use exact predicate equivalence when rewriting real SQL.

4. Work memory and sort spill

work_mem applies per sort/hash operation, and parallel workers may each allocate it. Use SET LOCAL in a transaction to test safely; shared_buffers is a start-time setting and should not be changed during this exercise.

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
BEGIN;
SET LOCAL work_mem = '64kB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, sum(amount) AS total
FROM lab_orders GROUP BY customer_id ORDER BY total DESC;
COMMIT;
BEGIN;
SET LOCAL work_mem = '16MB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, sum(amount) AS total
FROM lab_orders GROUP BY customer_id ORDER BY total DESC;
COMMIT;
SQL
```

Record Sort Method, Disk, Hash Batches, temp blocks and time. A larger value can reduce spill but may not improve wall time. For server sizing, inspect RAM with free -h and SHOW shared_buffers; test changes in a staging cluster before a Patroni-managed rolling restart.

5. WAL and checkpoint exercise

Keep fsync and full_page_writes enabled. Do not disable durability for a tuning demonstration. WAL bytes measure generated log; checkpoint counters show whether checkpoints are requested frequently. Parameter changes on a Patroni cluster belong in the dynamic configuration and can affect every node.

5.1 Observe WAL for a transaction

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
SELECT wal_records, wal_fpi, wal_bytes, wal_buffers_full FROM pg_stat_wal;
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
UPDATE lab_orders SET amount = amount + 1
WHERE customer_id BETWEEN 1 AND 100;
ROLLBACK;
SELECT wal_records, wal_fpi, wal_bytes, wal_buffers_full FROM pg_stat_wal;
SQL
```

A rollback still produces WAL and statistical counters can lag. Repeat a COMMIT only if the exercise requires a persistent write. Record replication lag before increasing workload.

5.2 Review checkpoint policy; optional cluster change

```
sudo -u postgres psql -d postgres -X <<'SQL'
SELECT name,setting,unit FROM pg_settings WHERE name IN
('max_wal_size','min_wal_size','checkpoint_timeout','checkpoint_completion_target','wal_compression');
SQL
sudo -u postgres /opt/patroni/bin/patronictl -c /etc/patroni/patroni.yml show-config postgres-ha
```

Only if the lab has spare disk and all replicas are healthy, demonstrate a modest cluster-wide max_wal_size change. Save the original value first, check disk with df -h, and verify all three members afterward:

```
df -h
sudo -u postgres /opt/patroni/bin/patronictl -c /etc/patroni/patroni.yml edit-config postgres-ha --pg \
max_wal_size=2GB
sudo -u postgres psql -d postgres -X -c "SHOW max_wal_size;"
sudo -u postgres /opt/patroni/bin/patronictl -c /etc/patroni/patroni.yml list
```

If the value was originally different, restore that exact original value with edit-config --pg max_wal_size=ORIGINAL_VALUE (substitute your recorded value). Higher max_wal_size permits more WAL between checkpoints but requires disk headroom; it is not a fixed disk cap. Never run CHECKPOINT merely to manufacture a favorable benchmark.

6. Autovacuum and table health

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
SELECT relname,n_live_tup,n_dead_tup,last_autovacuum,
       last_autoanalyze,autovacuum_count
FROM pg_stat_user_tables WHERE relname = 'lab_orders';
SELECT name,setting FROM pg_settings
WHERE name IN ('autovacuum','autovacuum_vacuum_scale_factor',
              'autovacuum_analyze_scale_factor','autovacuum_max_workers');
ALTER TABLE lab_orders SET (autovacuum_vacuum_scale_factor = 0.05,
                             autovacuum_analyze_scale_factor = 0.02);
UPDATE lab_orders SET amount = amount + 1
WHERE order_id BETWEEN 1 AND 10000;
VACUUM (ANALYZE, VERBOSE) lab_orders;
SELECT relname,n_dead_tup,last_vacuum,last_analyze

FROM pg_stat_user_tables WHERE relname = 'lab_orders';
SQL
```

Thresholds here apply only to lab_orders. VACUUM cannot run inside a transaction block. Statistics can update asynchronously. Avoid disabling autovacuum: it is needed for dead tuples and transaction ID wraparound protection.

7. Performance metrics: waits, locks and buffers

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
SELECT wait_event_type,wait_event,count(*) AS sessions
FROM pg_stat_activity
WHERE state <> 'idle' AND pid <> pg_backend_pid()
GROUP BY 1,2 ORDER BY sessions DESC;
SELECT a.pid,a.usename,a.state,a.wait_event_type,a.wait_event,
       pg_blocking_pids(a.pid) AS blocked_by, left(a.query,90) AS query
FROM pg_stat_activity a
WHERE cardinality(pg_blocking_pids(a.pid)) > 0;
SELECT locktype,mode,granted,count(*)
FROM pg_locks GROUP BY 1,2,3 ORDER BY 1,2,3;
SQL
```

A wait is an observation, not automatically a problem. For a hands-on lock demo, open session A on the primary and leave it in a transaction; run session B with a 3-second timeout, then COMMIT session A:

```
# Session A: sudo -u postgres psql -d perf_lab -X
BEGIN;
UPDATE lab_orders SET amount = amount + 1 WHERE order_id = 1;
# Keep this psql session open until session B completes.
```

```
# Session B: sudo -u postgres psql -d perf_lab -X
SET lock_timeout = '3s';
UPDATE lab_orders SET amount = amount + 1 WHERE order_id = 1;
SELECT wait_event_type,wait_event FROM pg_stat_activity
WHERE pid = pg_backend_pid();
# Session A: COMMIT;
```

While B waits, execute the blocked-by query above in a third session. The timeout is expected. In session B, ROLLBACK if its transaction becomes aborted.

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
SELECT datname,blks_hit,blks_read,
       round(100.0*blks_hit/nullif(blks_hit+blks_read,0),2)
       AS hit_pct, temp_bytes
FROM pg_stat_database WHERE datname = 'perf_lab';
SELECT backend_type,object,context,reads,hits,writes,fsyncs
FROM pg_stat_io WHERE object IN ('relation','wal')
ORDER BY backend_type,object,context LIMIT 25;
SQL
```

Hit ratio alone is not a universal target; include query latency and storage reads. Shared buffers are PostgreSQL cache, distinct from the OS page cache. pg_stat_io gives I/O by backend and context; NULL can mean the operation is not applicable.

8. PgBouncer verification and configuration

Run on pg-node-2, where PgBouncer already listens on 6432. Existing banking database mapping might point to the former leader: edit its backend host to the current primary before benchmarking. PgBouncer does not automatically discover a Patroni leader from a static IP.

```
sudo systemctl status pgbouncer --no-pager
sudo sed -n '/^\[databases\]/,/^\[pgbouncer\]/p' /etc/pgbouncer/pgbouncer.ini
sudo grep -E '^(pool_mode|listen_port|auth_type)= ' /etc/pgbouncer/pgbouncer.ini
sudo grep -E '^(default_pool_size|max_client_conn)= ' /etc/pgbouncer/pgbouncer.ini
sudo ss -ltnp | grep ':6432'
```

For the disposable database, add a perf_lab mapping using the current leader IP. This example assumes the leader is 10.10.0.13: replace it with the verified leader. Keep the existing banking line. Edit the [databases] section in sudo nano /etc/pgbouncer/pgbouncer.ini:

```
perf_lab = host=10.10.0.13 port=5432 dbname=perf_lab
```

Verify the selected PgBouncer login role exists in PostgreSQL, has CONNECT to perf_lab, and has a working SCRAM entry in /etc/pgbouncer/userlist.txt. For the existing bank_app login, run on the primary:

```
sudo -u postgres psql -d postgres -X -c "GRANT CONNECT ON DATABASE perf_lab TO bank_app;"
sudo -u postgres psql -d perf_lab -X -c "GRANT USAGE ON SCHEMA public TO bank_app;"
sudo -u postgres psql -d perf_lab -X -c "GRANT SELECT ON lab_orders TO bank_app;"
```

Run the three GRANT commands above on the primary. After editing the mapping, run sudo systemctl reload pgbouncer on pg-node-2. Never paste passwords or SCRAM verifiers into a shared manual. Prompt for the existing bank_app password when testing:

```
psql -h 127.0.0.1 -p 6432 -U bank_app -d perf_lab -c "SELECT count(*) FROM lab_orders;"
```

For admin metrics, connect to the special pgbouncer database using a configured admin_users or stats_users account, then execute SHOW POOLS and SHOW STATS. SHOW POOLS in perf_lab or banking is sent to PostgreSQL and fails.

```
psql -h 127.0.0.1 -p 6432 -U postgres -d pgbouncer -c "SHOW POOLS;"
psql -h 127.0.0.1 -p 6432 -U postgres -d pgbouncer -c "SHOW STATS;"
```

Transaction pooling is appropriate for short independent transactions. Check application use of session settings, temporary tables, LISTEN/NOTIFY, cursors and prepared statements before switching real traffic to transaction pooling.

9. Benchmark direct and pooled connections

Install the matching PostgreSQL client tools on the primary and pg-node-2 if pgbench is missing. On Ubuntu with PGDG PostgreSQL 18:

```
pgbench --version
sudo apt-get update
sudo apt-get install -y postgresql-client-18
```

Use a separate benchmark database, since pgbench -i creates its own tables. Run on the current primary:

```
sudo -u postgres createdb perf_bench
sudo -u postgres pgbench -i -s 5 perf_bench
sudo -u postgres pgbench -c 10 -j 2 -T 60 -P 10 -r perf_bench
```

Record tps (including/excluding connection establishment where shown), latency, errors and per-command timing. Repeat with c=30 to observe contention; never infer a gain from one run. To compare pooling on the same workload, grant permissions for bank_app on the pgbench objects or use a dedicated benchmark user with suitable privileges, then add perf_bench to PgBouncer [databases]. Avoid putting a password in the command or environment history. On pg-node-2, after editing/reloading the mapping, run:

```
pgbench -h 127.0.0.1 -p 6432 -U bank_app -c 10 -j 2 -T 60 -P 10 -r perf_bench
```

The default pgbench TPC-B-like workload needs SELECT/INSERT/UPDATE privileges on pgbench_accounts, pgbench_branches, pgbench_tellers, pgbench_history, plus USAGE on their sequences. Execute on the primary after initialization:

```
sudo -u postgres psql -d postgres -X <<'SQL'
GRANT CONNECT ON DATABASE perf_bench TO bank_app;
SQL
sudo -u postgres psql -d perf_bench -X <<'SQL'
GRANT USAGE ON SCHEMA public TO bank_app; GRANT SELECT,INSERT,UPDATE ON ALL TABLES IN SCHEMA public TO bank_app; GRANT USAGE,SELECT ON ALL SEQUENCES IN SCHEMA public TO bank_app;
SQL
```

Direct and pooled benchmarks must use comparable network paths to attribute differences to pooling. The local Unix socket direct run versus TCP on pg-node-2 is an instructional comparison, not a controlled proof of pooling improvement. For a comparable run, execute both against the same primary and PgBouncer endpoints from the same client VM with the same user and options.

```
pgbench -h 10.10.0.13 -p 5432 -U bank_app -c 10 -j 2 -T 60 -P 10 -r perf_bench
pgbench -h 10.10.0.12 -p 6432 -U bank_app -c 10 -j 2 -T 60 -P 10 -r perf_bench
```

Replace 10.10.0.13 with the current leader and ensure perf_bench is mapped on PgBouncer. Monitor patronictl list, pg_stat_wal and pg_stat_checkpointed during each run. PgBouncer generally helps connection-heavy workloads; a small fixed pool of persistent pgbench sessions may show little gain.

10. Generate load, apply tuning, and measure again

This is the main before/after lab. Run on the current primary. Use the SAME VM, data, client count, script, duration and connection path in each pair. Do one practice run first to warm caches, then collect at least three measured runs if time permits. Run one benchmark at a time. The query exercises use read-only pgbench scripts; the default pgbench workload in step 9 generates writes.

10.1 Index experiment: start without the customer index

The index from step 3 would make the baseline already tuned. Remove only the exercise index, then generate repeatable SELECT traffic with changing customer IDs. Keep the date index for the separate range exercise.

```
sudo -u postgres psql -d perf_lab -X -c 'DROP INDEX IF EXISTS lab_orders_customer_date_idx;'
cat > /tmp/day6_customer.sql <<'SQL'
\set cid random(1,10000)
```

```
SELECT * FROM lab_orders
WHERE customer_id = :cid AND status = 'PAID'
ORDER BY order_date DESC LIMIT 20;
SQL
sudo -u postgres pgbench -n -c 8 -j 2 -T 15 -f /tmp/day6_customer.sql perf_lab
```

Capture the counters immediately before and after each measured run. These cumulative values are useful as deltas; do not reset production statistics. Use the same command before and after the baseline, then repeat around the tuned run:

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
SELECT now(),blks_hit,blks_read,temp_bytes
FROM pg_stat_database WHERE datname='perf_lab';
SELECT now(),wal_records,wal_bytes FROM pg_stat_wal;
SQL
sudo -u postgres pgbench -n -c 8 -j 2 -T 60 -P 10 -f /tmp/day6_customer.sql perf_lab | tee \
/tmp/day6_index_before.txt
```

Record tps and latency from the summary. Apply the tuning change, update planner statistics, and confirm the execution plan:

```
sudo -u postgres psql -d perf_lab -X <<'SQL'
CREATE INDEX lab_orders_customer_date_idx
ON lab_orders (customer_id,order_date DESC);
ANALYZE lab_orders;
EXPLAIN (ANALYZE,BUFFERS) SELECT * FROM lab_orders
WHERE customer_id=427 AND status='PAID'
ORDER BY order_date DESC LIMIT 20;
SQL
sudo -u postgres pgbench -n -c 8 -j 2 -T 60 -P 10 -f /tmp/day6_customer.sql perf_lab | tee \
/tmp/day6_index_after.txt
```

Interpretation: higher TPS and lower latency support an improvement for this workload. Check buffer reads and the scan node as well. An index is not free: review disk size and test write-heavy workload separately before recommending it broadly.

10.2 Memory experiment: force a sort, then reduce spills

Generate concurrent aggregate-and-sort traffic. The two runs differ only in session work_mem. If 64kB spills, compare Sort Method/Hash Batches and temp_bytes with 16MB. These values are demonstration settings, not recommended defaults.

```
cat > /tmp/day6_sort.sql <<'SQL'
SELECT customer_id,sum(amount) AS total
FROM lab_orders GROUP BY customer_id ORDER BY total DESC LIMIT 100;
SQL
sudo -u postgres env PGOPTIONS='-c work_mem=64kB' pgbench -n -c 4 -j 2 -T 30 -f /tmp/day6_sort.sql \
perf_lab | tee /tmp/day6_mem_before.txt
sudo -u postgres env PGOPTIONS='-c work_mem=16MB' pgbench -n -c 4 -j 2 -T 30 -f /tmp/day6_sort.sql \
perf_lab | tee /tmp/day6_mem_after.txt
```

Use the EXPLAIN queries in step 4 to verify spill before and after. pg_stat_database.temp_bytes is cumulative; capture it around each run for a meaningful delta. With four clients, several plan nodes or parallel workers may allocate work_mem independently.

10.3 WAL and checkpoint experiment under write load

Use the perf_bench database created in step 9. Collect the following before and after each 60-second default pgbench run. Run it first with the original Patroni settings. If you changed max_wal_size in step 5, repeat with the changed value, then restore the original value. A single minute may produce zero checkpoints; extend the test only if disk and replica lag allow.

```
sudo -u postgres psql -d postgres -X <<'SQL'
SELECT now(),wal_bytes,wal_records,wal_fpi
FROM pg_stat_wal;
SELECT now(),num_timed,num_requested,num_done
FROM pg_stat_checkpoint;
SQL
sudo -u postgres pgbench -c 10 -j 2 -T 60 -P 10 perf_bench | tee /tmp/day6_wal_run.txt
sudo -u postgres /opt/patroni/bin/patronictl -c /etc/patroni/patroni.yml list
```

Write down the counter deltas, latency, TPS, disk free space and replica lag for each run. `max_wal_size` changes checkpoint frequency, not the number of logical writes. Do not expect a throughput gain unless checkpoints were a bottleneck. Keep `fsync` and `full_page_writes` enabled throughout.

10.4 Results sheet and decision

Complete one row per measured run: scenario | clients | seconds | TPS | average latency (ms) | buffer reads delta | temp bytes delta | WAL bytes delta | requested checkpoints delta | replica lag. Improvement % for TPS = $100 \times (\text{tuned TPS} - \text{baseline TPS}) / \text{baseline TPS}$; for latency report the percentage decrease. If results vary, repeat and compare the median. State whether the change is worth its memory, disk and write cost.

11. Wrap-up and cleanup

Document before/after plan, execution time, temp spill, WAL bytes, checkpoint counts, TPS and latency. Check that all Patroni replicas are streaming. Restore any optional cluster setting to its recorded original value. Remove the added PgBouncer mappings and grants when no longer needed; drop only the disposable lab databases.

```
sudo -u postgres /opt/patroni/bin/patronictl -c /etc/patroni/patroni.yml list
sudo -u postgres dropdb perf_bench
sudo -u postgres dropdb perf_lab
rm -f /tmp/day6_customer.sql /tmp/day6_sort.sql
```

Checkpoint questions: Why did the planner choose a scan? How much did the index add to write cost? Did higher `work_mem` reduce spills? Which wait event appeared during the blocked UPDATE? How did WAL bytes and requested checkpoints change? Which client path was used in each benchmark?

Official references

<https://www.postgresql.org/docs/18/sql-explain.html>

<https://www.postgresql.org/docs/18/runtime-config-resource.html>

<https://www.postgresql.org/docs/18/runtime-config-wal.html>

<https://www.postgresql.org/docs/18/runtime-config-vacuum.html>

<https://www.postgresql.org/docs/18/monitoring-stats.html>

<https://www.postgresql.org/docs/18/pgbench.html>

https://patroni.readthedocs.io/en/latest/patroni_configuration.html

<https://www.pgбouncer.org/config.html>